

Practice 3 - BFS

Deadline: Mar.25/26/27. Should be checked onsite (during labs).Online checks via email or messaging apps are not accepted.

This lab reinforces your understanding of **Breadth-First Search (BFS)** through two practical applications from the lecture:

1. **Flood Fill:** Simulate the "paint bucket" tool used in image editing software.
 2. **Bipartite Graph Testing:** Determine if a graph is bipartite.
-

Task 1: Flood Fill Simulation(0.5 points)

Goal: Implement the "paint bucket" tool using BFS.

Problem:

- Given an $R \times C$ grid where each cell has a color (integer)
- Given a starting pixel (sr, sc) (0-indexed) and a `newColor`
- Recolor the entire connected region of the starting pixel's **original color** to `newColor`

Input Format:

- The first line, two integers R and C denoting the number of rows and columns;
- The next R lines, each containing C integers representing the color of each cell in the grid;
- The last line, three integers sr , sc , and `newColor` denoting the starting position and the target color.

Output Format:

- The output consists of R lines, each containing C integers representing the modified grid.

Sample Input 1:

```
3 3
1 1 1
1 1 0
1 0 1
0 0 2
```

Expected Output 1:

```
2 2 2
2 2 0
2 0 1
```

*Explanation: Starting from (0,0) with original color 1, all connected 1's are changed to 2. **Sample Input 2:***

```

4 5
1 1 1 1 1
1 1 0 1 1
1 0 0 0 1
1 1 1 1 1
1 2 4

```

Expected Output 2:

```

1 1 1 1 1
1 1 4 1 1
1 4 4 4 1
1 1 1 1 1

```

Sample Input 3:

```

3 3
1 1 1
1 0 1
1 1 1
2 2 2

```

Expected Output 3:

```

2 2 2
2 0 2
2 2 2

```

Task 2: Bipartite Graph Testing (0.5 points)

Goal: Use BFS to determine if an undirected graph is bipartite.

Problem:

- Given an undirected graph with n nodes and m edges
- Determine if the graph is bipartite

Input Format:

- The first line, two integers n and m denoting the number of nodes and edges;
- The next m lines, each containing two integers u and v representing an edge between node u and node v . The nodes are indexed from 0 to $n-1$.

Output Format:

- The output consists of a single line containing **YES** if the graph is bipartite, or **NO** otherwise.

Sample Input 1:

```
4 4
0 1
1 2
2 3
3 0
```

Expected Output 1:

```
YES
```

Sample Input 2:

```
3 3
0 1
1 2
2 0
```

Expected Output 2:

```
NO
```

Sample Input 3:

```
6 7
0 1
1 2
2 3
3 4
4 5
5 0
1 4
```

Expected Output 3:

```
YES
```

Sample Input 4:

```
17 14
0 1
1 2
2 0
3 4
4 5
5 6
6 3
7 8
8 9
11 12
11 11
13 14
14 15
15 16
```

Expected Output 4:

NO

Sample Input 5:

```
17 14
0 1
1 2
2 3
3 4
4 5
5 6
6 3
7 8
8 9
11 12
12 11
13 14
14 15
15 16
```

Expected Output 5:

YES

Submission

Show the following to your tutor or SA:

1. **Source Code**
2. **Sample Run Results**

Grading:

- Submissions on time: 0.5-1.0 points
- Late submissions: 0 points.

In-Class Self-Check List Before submitting, please review this checklist to ensure you meet the in-class assessment criteria:

- **Compilation:** The code compiles successfully without errors.
 - **Execution:** The program must generate correct results and complete execution within a reasonable time limit.
 - **Code Understanding:** show that you understand every line of your code, not just copy from somewhere.
-